

— SOFTWARE REQUIREMENTS SPECIFICATION

PERSONAL WORKOUT TRACKER

A menu-driven CLI application for logging workouts, tracking exercise progression, and analyzing performance trends — built with Java OOP, Spring Boot, and MongoDB.

JAVA OOP

SPRING BOOT

MONGODB

MENU-DRIVEN

NO WEB LAYER

3

COLLECTIONS

3

EXERCISE SUBTYPES

5

SUBMENUS

3

ALERT STRATEGIES

Application Summary

The Personal Workout Tracker is a menu-driven command-line application built with Java, Spring Boot, and MongoDB. You maintain a personal exercise library — a catalogue of exercises you define with name, muscle group, and equipment. From there you log workout sessions, recording sets, reps, and weight per exercise. Over time the app tracks your progression on each exercise and generates performance reports showing volume trends, strength gains, and weekly and monthly summaries.

The navigation follows the same two-level numbered menu pattern established in the Finance and Warehouse projects — a main menu dispatches to submenus, each focused on one domain area. The architecture uses the three-collection pattern: users, exercises, and sessions — mirroring the accounts, products, and movements structure you already know well.

Technology Stack

Spring Boot

Entry point via `CommandLineRunner`. No web layer. Spring DI wires all services. `SessionContext` holds the current logged-in user.

Spring Data MongoDB

Three repositories: `UserRepository`, `ExerciseRepository`, `SessionRepository`. Polymorphic `_class` discriminator for exercise subtypes. Sessions reference exercises by `exerciseId`.

Java OOP

Abstract `Exercise` with three concrete subtypes each overriding `calculateVolume()`. Factory pattern for exercise creation. Chain of responsibility for alerts. Menu classes are Spring `@Component` beans wired via DI.

Menu System

Two-level numbered menus. `Scanner`-based `InputHandler` with debug mode. `do-while` loop navigation. `0` always means Back or Exit.

BCrypt

Passwords hashed via Spring Security Crypto. Plain-text passwords are never stored or logged.

Exercise Hierarchy

All exercise types extend the abstract `Exercise` class. The key differentiator is `calculateVolume()` — each subtype computes volume differently, giving a meaningful intensity metric for reports and progression tracking.

Exercise ABSTRACT <code>calculateVolume(SessionEntry)</code> · <code>getExerciseType()</code> · <code>getSummary()</code>		
StrengthExercise	CardioExercise	BodyweightExercise
— <code>calculateVolume(entry)</code> = sets × reps × weightKg	— <code>calculateVolume(entry)</code> = distanceKm × 1000	— <code>calculateVolume(entry)</code> = sets × reps × (bodyWeight + addedWeightKg)
— Sparkline tracks max weight over time	— Sparkline tracks best pace over time	— Sparkline tracks max reps over time
— e.g. bench press, squat, deadlift	— e.g. running, cycling, rowing	— e.g. pull-ups, push-ups, dips

Core Classes

User	id, name, username, hashedPassword, role (enum), bodyWeightKg, weeklySessionGoal. <code>bodyWeightKg</code> is needed by <code>BodyweightExercise.calculateVolume()</code> and stored on the user rather than on each session entry. Top-level MongoDB document.
Exercise (abstract)	id, userId, name, muscleGroup (enum), equipment (enum), exerciseType (enum), notes. Abstract methods: <code>calculateVolume()</code> , <code>getExerciseType()</code> . Own MongoDB collection — each user maintains their own exercise library. Subtypes stored via <code>_class</code> discriminator.
StrengthExercise	Extends Exercise. Adds no extra definition fields — sets, reps, and weight are recorded per session entry, not on the exercise definition itself.
CardioExercise	Extends Exercise. Adds no extra definition fields — duration and distance are recorded per session entry.
BodyweightExercise	Extends Exercise. Adds no extra definition fields — sets, reps, and optional added weight are recorded per session entry.
WorkoutSession	id, userId, sessionDate, durationMinutes, notes, List<SessionEntry>. Own MongoDB collection. Immutable once completed. <code>SessionEntry</code> is embedded — not a top-level document.
SessionEntry	Embedded in WorkoutSession. <code>exercisId</code> , <code>exerciseName</code> (denormalized for display), <code>exerciseType</code> , sets, reps, weightKg, durationMinutes, distanceKm, addedWeightKg, notes. Fields are nullable — only the relevant ones are populated per exercise type.

AlertContext	Carries data for alert evaluation: sessionEntry, workoutSession. Fields nullable — strategies check <code>supports()</code> to determine applicability. Transient — never persisted.
AlertManager	Spring <code>@Component</code> . Auto-populated <code>List<AlertStrategy></code> . Single <code>evaluate(AlertContext)</code> traverses all strategies. Called after each session entry is logged.

Enumerations

Role

ADMINISTRATOR	USER
---------------	------

ExerciseType

STRENGTH	CARDIO	BODYWEIGHT
----------	--------	------------

MuscleGroup

CHEST	BACK	SHOULDERS	BICEPS	TRICEPS	LEGS	CORE	FULL_BODY	CARDIO
-------	------	-----------	--------	---------	------	------	-----------	--------

Equipment

BARBELL	DUMBBELL	CABLE	MACHINE	BODYWEIGHT	TREADMILL	BIKE	ROWER	OTHER
---------	----------	-------	---------	------------	-----------	------	-------	-------

Design Requirements

INHERITANCE	INTERFACES	ENCAPSULATION	PATTERNS
· Abstract Exercise — 3 concrete subtypes	· AlertStrategy — supports(), evaluate()	· No direct field access	· Chain of Responsibility — AlertManager
· Each overrides calculateVolume()	· IMenu — show(), printOptions()	· Session immutable once completed	· Factory — ExerciseFactory
· Each overrides getExerciseType()	· Summarizable — getSummary()	· Volume calculated not stored	· Template Method — menu loop
· Constructor chaining via super()		· Password BCrypt hashed only	

Alert Strategies

OvertrainingStrategy	<code>supports()</code> — true when <code>context.getWorkoutSession() != null</code> . <code>evaluate()</code> — fires when the same primary muscle group has been trained on consecutive days. Warns the user that the
----------------------	--

muscle group may need rest. Called from `SessionService` when a session is completed.

GoalProgressStrategy

`supports()` — true when `context.getWorkoutSession() != null`.
`evaluate()` — fires when weekly session count drops below the user's configured target (default 3 sessions per week). Called from `ReportService` during report generation.

Key Design Notes

Exercise vs Entry

The `Exercise` document in the exercise library defines what the exercise is — name, muscle group, equipment, type. The `SessionEntry` embedded in a session records what you actually did — sets, reps, weight, distance. An exercise can appear in hundreds of sessions; the entry captures the performance for that specific session only.

Denormalized Name

`SessionEntry` stores `exerciseName` alongside `exerciseId`. This avoids a lookup when displaying session history — the name is already there. If an exercise is renamed the old session entries keep the original name, which is actually desirable for historical accuracy.

Volume Calculation

`calculateVolume()` is abstract on `Exercise` but is actually called on the `SessionEntry` via the resolved exercise subtype. `SessionService` loads the exercise by id, casts to the correct subtype, and calls `calculateVolume(entry)` passing the entry's recorded values. Volume is never stored — always derived.

ExerciseFactory

`ExerciseFactory.create(ExerciseType, ...)` prompts common fields (name, muscleGroup, equipment, notes) then type-specific ones. Since the exercise definition has no type-specific fields, the factory difference is only in which `Exercise` subtype is instantiated. The type determines how session entries are recorded in the session menus.

BodyweightExercise Volume

`BodyweightExercise.calculateVolume()` needs the user's body weight. This is fetched from `SessionContext.getCurrentUser().getBodyWeightKg()` at calculation time. If the user has not set their body weight, volume defaults to sets × reps only (added weight still applied if present).

Main Menu

PERSONAL WORKOUT TRACKER // V1.0		
1	Exercises	manage your library
2	Sessions	log and view workouts
3	Progress	exercise progression and trends
4	Reports	volume and frequency
5	Settings	account and preferences
0	Exit	
> Select option: █		

Submenus

1. EXERCISES	2. SESSIONS	3. PROGRESS
1 List all exercises	1 Start new session	1 Top exercises
2 Add exercise	2 View recent sessions	2 Exercise progression
3 View exercise detail	3 View session detail	3 Volume over time
4 Delete exercise	4 Delete session	0 Back
0 Back	0 Back	
4. REPORTS	5. SETTINGS	
1 Weekly summary	1 Update body weight	
2 Monthly summary	2 Set weekly session goal	
3 Muscle group frequency	3 Change password	
0 Back	4 Add user (<i>admin only</i>)	
	5 Delete user (<i>admin only</i>)	
	0 Back	

05 / FEATURES	FEATURE SPECIFICATIONS
---------------	------------------------

Exercises

List all exercises	Display all exercises in the library: name, type badge (STR / CRD / BWt), muscle group, equipment. Filterable by muscle group or equipment at the prompt.
Add exercise	Select exercise type first. <code>ExerciseFactory</code> instantiates correct subtype. Prompt name, muscle group, equipment, and optional

notes. Persist via `ExerciseRepository`.

View exercise detail Full definition fields plus last 5 session entries for this exercise and a mini ASCII sparkline () of the key metric over the last 8 sessions via `SparklineUtil.renderLastN(values, 8)`.

Delete exercise Confirm before deletion. Reject if any session entries reference this exercise — historical integrity must be preserved.

Sessions

Start new session Creates a new `WorkoutSession` with today's date. Prompts to add exercises one by one. For each exercise, prompts type-specific fields (sets/reps/weight for Strength, duration/distance for Cardio, sets/reps/added weight for Bodyweight). After each entry, calls `alertManager.evaluate()` to check for goal alerts. Ends when user enters 0. Prompts for overall session duration and notes before saving.

View recent sessions Last 10 sessions sorted by date descending. Shows date, duration, number of exercises, and total session volume.

View session detail Full session detail: date, duration, notes, and each entry with exercise name, recorded values, and calculated volume.

Delete session Confirm before deletion.

Progress

Top exercises List the top 10 most logged exercises by session entry count, showing total entries, total volume, and average volume per entry. Sorted by entry count descending.

Exercise progression Select an exercise and a date range. Display a table of every session entry for that exercise — date, recorded values, calculated volume — and an ASCII sparkline () of the key metric over time. Key metric is determined by exercise type: `weightKg` for Strength, `durationMinutes ÷ distanceKm` (pace) for Cardio, `reps` for Bodyweight.

Volume over time Total weekly training volume (sum of all entry volumes) plotted as an ASCII bar chart across the last 8 weeks. Highlights weeks where volume increased vs decreased.

Reports

Weekly summary Current week: sessions completed vs goal, total volume, most trained muscle group, and longest session (highest total volume

single workout). Triggers `GoalProgressStrategy` alert if behind on session target.

Monthly summary Selected month: total sessions, total volume, volume by exercise type (Strength / Cardio / Bodyweight), average session volume (total volume ÷ sessions completed), and top 3 most logged exercises.

Muscle group frequency How many times each muscle group was trained in a selected period. ASCII bar chart sorted by frequency. Highlights imbalances — e.g. chest trained 8x but back only 2x.

Settings

Options 4 and 5 are only displayed when `Role.ADMINISTRATOR.equals(user.getRole())`. The service layer additionally enforces this guard — `UserService.addUser()` and `UserService.deleteUser()` verify the current user's role via `SessionContext` and throw an `UnauthorizedException` if called by a non-administrator.

Update body weight Prompts for new body weight in kg. Updates the current user's `bodyWeightKg` field. Used by `BodyweightExercise.calculateVolume()`.

Set weekly session goal Prompts for target sessions per week (default 3). Stored on the user. Referenced by `GoalProgressStrategy` during weekly report generation.

Change password Prompts for current password (verified), then new password twice. Hashed via Spring Security Crypto before persisting.

Add user (*admin only*) Prompts for name, username, and initial password. Role defaults to USER. Persists via `UserRepository`. Only displayed and executable by ADMINISTRATOR.

Delete user (*admin only*) Select a user from the list. Confirm before deletion. Cannot delete own account. Only displayed and executable by ADMINISTRATOR.

Session Entry Flow

When logging a session entry the prompts adapt to the exercise type so the user is never asked irrelevant fields:

```
Adding exercise to session...
Search exercise: bench
> 1) Bench Press [STR · CHEST · BARBELL]
    2) Incline Bench [STR · CHEST · DUMBBELL]
Select: 1
```

Sets: 4
Reps: 8
Weight (kg): 100

Volume: 3200 kg

Add another exercise? (Y/0 to finish):

Package Structure

io.workout

```
├─ WorkoutApplication.java           // Spring Boot entry point
├─ menu/
│   ├─ MainMenu.java
│   ├─ ExerciseMenu.java
│   ├─ SessionMenu.java
│   ├─ ProgressMenu.java
│   ├─ ReportMenu.java
│   └─ SettingsMenu.java
├─ model/
│   ├─ User.java
│   ├─ Exercise.java                 // abstract
│   ├─ StrengthExercise.java
│   ├─ CardioExercise.java
│   ├─ BodyweightExercise.java
│   ├─ WorkoutSession.java
│   ├─ SessionEntry.java             // embedded in WorkoutSession
│   ├─ AlertContext.java             // transient
│   └─ AlertManager.java
├─ enums/
│   ├─ Role.java
│   ├─ ExerciseType.java
│   ├─ MuscleGroup.java
│   └─ Equipment.java
├─ interfaces/
│   ├─ IMenu.java
│   ├─ AlertStrategy.java
│   └─ Summarizable.java
├─ alert/
│   ├─ OvertrainingStrategy.java
│   └─ GoalProgressStrategy.java
└─ factory/
```

```

|   └─ ExerciseFactory.java
|   └─ exception/
|       └─ EntityNotFoundException.java
|       └─ ExerciseInUseException.java
|   └─ repository/
|       └─ UserRepository.java
|       └─ ExerciseRepository.java    // findById(), findByIdAndMuscleGroup(), findByIdAnd
|       └─ SessionRepository.java    // findByIdOrderByDateDesc(), findTop10()
|   └─ service/
|       └─ UserService.java
|       └─ ExerciseService.java
|       └─ SessionService.java
|       └─ ReportService.java
|   └─ authentication/
|       └─ AuthenticationService.java
|       └─ SessionContext.java        // holds current User
|   └─ util/
|       └─ InputHandler.java          // debug mode reads from test_input.txt
|       └─ SparklineUtil.java         //  ASCII chart renderer
|       └─ BarChartUtil.java          // ASCII bar chart for volume reports
|       └─ CommandLineTable.java

```

MongoDB Document Mapping

COLLECTION	AGGREGATE ROOT	EMBEDDED	NOTES
users	User	—	Single user type. Stores bodyWeightKg for volume calculation and weeklySessionGoal for alerts.
exercises	Exercise (+ subtypes)	—	Polymorphic via <code>_class</code> . Exercise library — defines what exercises exist, not performance data.
sessions	WorkoutSession	List<SessionEntry>	Session entries embedded — they always load together. exerciseId references exercises collection. exerciseName denormalized for display.

Sample Data

TYPE	NAME	CATEGORY	KEY FIELDS
STRENGTH	Bench Press	CHEST · BARBELL	Classic compound push — good for testing PR detection
STRENGTH	Back Squat	LEGS · BARBELL	Primary lower body — triggers overtraining alert if done daily
STRENGTH	Deadlift	BACK · BARBELL	Full body pull — high volume metric
CARDIO	Treadmill Run	CARDIO · TREADMILL	Tests pace PR detection and distance tracking
CARDIO	Rowing Machine	CARDIO · ROWER	Full body cardio — distance based volume
BODYWEIGHT	Pull-ups	BACK · BODYWEIGHT	Tests added weight optional field — max reps PR
BODYWEIGHT	Push-ups	CHEST · BODYWEIGHT	No added weight — bodyWeight only in volume calc
SESSION	Push Day	3 exercises	Bench Press × 4×8×100kg, Overhead Press × 3×10×60kg, Push-ups × 3×20
SESSION	Pull Day	3 exercises	Deadlift × 3×5×140kg, Pull-ups × 4×10, Rowing Machine × 20min/5km